A 6-DOF Simulation and Multi-Objective Reinforcement Learning Framework for Autonomous Rocket Booster Landing

Rohan James and Ivana Dusparic

Trinity College Dublin, the University of Dublin, Ireland
{jamesro,duspari}@tcd.ie

Abstract. Autonomous landing of rocket boosters is among the most demanding control problems in aerospace engineering, requiring the simultaneous optimisation of competing objectives – fuel efficiency, positional accuracy, and orientation stability – subject to constraints such as gimbal angle limits and glideslope boundaries. Conventional guidance systems based on convex optimisation and model predictive control require accurate system models and struggle to generalise across diverse uncertainties. Reinforcement Learning (RL) offers a model-free alternative, demonstrating success in 3 degrees-of-freedom (DOF) and 6-DOF landing simulations. However, prevailing approaches scalarise the multi-objective landing problem into a single reward signal. This requires manual, arbitrary weight tuning that obscures trade-offs and prevents recovery of diverse Pareto-optimal policies from a single training run. Multi-objective RL (MORL) addresses this by explicitly learning a Pareto front, allowing operators to prioritise objectives at deployment, yet its application to rocket landing remains largely unexplored. This work presents, to our knowledge, the first MORL approach optimising the full 6-DOF rocket landing dynamics, evaluated in custom PyBullet simulation wrapped in an OpenAI Gym interface. Single-objective RL achieves a landing rate of up to 93.0% (95% CI $\pm$ 1.6); MORL achieves a comparable 92.9% ($\pm$ 1.6) under a precision-favoured weighting, with landing rate varying across preference weights (83.4% under a fuel-favoured weighting), and yields a Pareto front of ten non-dominated policies balancing landing precision against fuel usage.

Keywords: Reinforcement Learning · Pareto Optimisation · Multi Objective Reinforcement Learning · Aerospace Guidance and Control · 6-DOF Simulation · Autonomous Rocket Booster Landing

1 Introduction

Reusable rocket boosters have altered space economics; SpaceX's Falcon 9 and Blue Origin's New Glenn show that recovering and reconditioning first-stage boosters, rather than discarding them after a single flight, can dramatically reduce the cost per kilogram to orbit [18]. This depends critically on precise

autonomous vertical landing, one of the most complex control tasks in modern aerospace engineering. The vehicle must counter aerodynamic disturbances, manage limited propellant, and reject environmental perturbations, all in real-time within a restricted target zone, without human intervention. State-of-the-art guidance architectures rely on convex optimisation, Model Predictive Control (MPC), and computer vision to generate near-optimal trajectories on board [21]. These model-based approaches achieve high success rates in nominal conditions but remain sensitive to systematic uncertainties, and performance can deteriorate under component degradation, hardware faults, or unmodelled dynamics.

The landing problem is inherently multi-objective: minimising fuel consumption directly conflicts with the aggressive attitude corrections needed for precision, and these objectives cannot be meaningfully collapsed into a single metric without the aid of implicit weighting schemes. Reinforcement Learning (RL) offers a compelling model-free alternative for this class of problems [19], but the dominant literature reduces landing to a single scalarised reward signal, obscuring the true trade-off structure and yielding one policy that may not suit varying mission requirements [10, 7]. Multi-Objective Reinforcement Learning (MORL) instead treats each objective as a distinct reward signal and learns a set of policies characterising the Pareto front of achievable trade-offs [12]. Despite success across other autonomous systems, MORL's application to six-degrees-of-freedom (6-DOF) rocket landing remains largely uninvestigated. This is a natural consequence of realistic rocket simulation environments being *commercially sensitive and typically proprietary* and public alternatives such as Lunar-Lander-v3 and MO-Lunar-Lander *not modelling the full 6-DOF rocket dynamics* [20, 8].

This paper addresses this gap with three contributions. First, we build a 6-DOF rocket landing environment in PyBullet, modelling full translational and rotational dynamics with thrust vectoring and physical landing constraints, wrapped in an OpenAI Gym interface for reproducibility and compatibility with standard RL libraries. Second, we present, to our knowledge, the first formalisation of realistic 6-DOF rocket booster landing as a multi-objective Markov Decision Process (MOMDP), explicitly capturing the landing precision versus propellant efficiency trade-off. Third, we provide a comparative evaluation of a classical proportional-derivative (PD) controller, single-objective baselines (PPO, SAC), and a MORL algorithm (CAPQL), demonstrating that MORL recovers a diverse family of Pareto-optimal policies where single-objective RL (SORL) yields only one. The environment and code for this work are available for review at: `https://github.com/rohan-james/6dof-rocket-booster-lander-morl`.

2 Related Work

2.1 Methods of Rocket Landing Control

Autonomous rocket booster landing, as demonstrated operationally by SpaceX and Blue Origin, has historically relied on model-based guidance. The Proportional-Integral-Derivative (PID) controller remains a foundational component of many

guidance, navigation and control (GNC) systems for its simplicity and tunability [2]. It primarily performs trajectory error correction in real time from the current, accumulated, and rate-of-change error terms. MPC instead solves a constrained optimisation problem over a short prediction horizon at each timestep, re-planning the control sequence as new state estimates arrive [5]. It is frequently paired with convex optimisation techniques, such as lossless and successive convexification, to generate globally optimal trajectories in real time [1, 21]. These model-based approaches achieve high precision under nominal, well-parametrised dynamics but depend on accurate system models and can degrade under disturbances, unmodelled dynamics, or hardware faults, motivating model-free, learning-based alternatives such as RL.

RL offers a model-free alternative to the guidance schemes described in Section 1, learning policies directly from simulated interactions rather than from an explicit dynamics model. Early work targeted simplified two-dimensional benchmarks: on an extended OpenAI Gym LunarLander, DQN generally outperformed discretised SARSA under observation noise and engine failures [9], while on a custom eight-dimensional variant, a simple heuristic and tabular Q-learning outperformed DQN and DDQN [16]. This suggested that low-fidelity environments do not require sophisticated agents. At higher fidelity, a curriculum-trained TD3 agent first stabilising attitude, and then optimising position and fuel economy, converged under wind and thrust disturbances where standard TD3 failed [22]; in a 3-DOF Box2D environment, PPO achieved stable, precise landings against DDPG's 35.9% accuracy [7]; and, an Adaptive PPO variant with clipping and phase-aware normalisation improved both convergence speed and landing rate over standard PPO (96.8% vs. 78.2%) [23]. At full 6-DOF, a closed-loop PPO problem with reward shaped to align velocity with the line-of-sight vector has been used for rocket landing [10], while a model-based RL framework, Harmony-TDMPC2, achieved an 82.1% success rate under stochastic 6-DOF dynamics [3]. Across the RL literature, two limitations recur. First, many studies rely on 2D or reduced-order dynamics and second, competing mission objectives are collapsed into a single scalarised reward, concealing the underlying trade-offs and yielding one policy per training run rather than a family of options. These limitations motivate framing rocket landing explicitly as a multi-objective problem.

2.2 Multi-Objective Reinforcement Learning

Multi-Objective Reinforcement Learning (MORL) modifies the standard RL formulation by replacing the scalar reward with a vector-valued reward $\mathbf{r} \in \mathbb{R}^n$, in which each component encodes a distinct, potentially conflicting objective. Where conventional single-objective RL converges to one optimal policy with fixed *a priori* weighting of these objectives, MORL instead seeks to approximate the *Pareto front* - the set of policies for which no objective can be improved without degrading another, deferring the choice of preference to deployment time rather than fixing it before training begins. Hayes et al. [12] argue this is not merely a practical convenience but a formal necessity. Unless a decision maker's utility function is exactly known in advance, any fixed scalarisation reaches only

a subset of achievable trade-offs, and when preferences are non-linear, *a priori* scalarisation is incapable of recovering the full solution set.

Among MORL algorithms, Concave-Augmented Pareto Q-Learning (CAPQL) *extends* the maximum-entropy, off-policy actor-critic framework of Soft Actor-Critic (SAC) to the multi-objective setting by conditioning the policy and vector-valued critics on a sampled preference vector, and augmenting the reward with a concave regularisation term that stabilises training against abrupt preference shifts [15]. Unlike linear-scalarisation approaches, which recover only the convex coverage set, CAPQL is designed to reach non-convex regions of the Pareto front while remaining a single, preference-conditioned policy rather than an ensemble of independently trained ones.

MORL has been applied across several autonomous domains involving conflicting objectives. Deep W-Networks extended W-learning with DQN to resolve competing objectives via per-objective value estimates, showing performance broadly comparable to DQN with improved stability [14]. The same architecture was applied to self-adaptive web server configuration, trading convergence speed for lower response times [17]. Finally, a constrained MORL framework for autonomous highway merging achieved hypervolume improvements exceeding 950% over classical baselines in three-objective settings [13].

3 Custom 6-DOF Rocket Booster Environment

Existing open-source landing environments, such as `LunarLander-v3` [20], model only planar 3-DOF dynamics and do not capture the coupled rotational translational behaviour of a powered vertical descent. To address this gap, we develop a custom rocket landing environment in PyBullet [6], that models all six degrees of freedom: translation in *x, y, z* and rotation in *roll, pitch, and yaw*. PyBullet was selected for its direct support for rigid-body force and torque application and its lower computational overhead relative to MuJoCo or Gazebo; the environment is wrapped in the Gymnasium interface.

The booster is modelled as a uniform solid cylinder (Table 1), actuated by one gimballed engine mounted on the centreline, $\ell = 1.5\,\mathrm{m}$ below the centre of mass. Mass and thrust are scaled down to keep the dynamics comparable to facilitate shorter episodes, thereby reducing training cost; the booster is a reduced-scale surrogate. The action space is a three-dimensional continuous vector $\mathbf{a} \in [-1, 1]^3$ commanding throttle and two gimbal deflections. Throttle maps to a thrust magnitude $T \in [0, T_{\max}]$; each gimbal command maps to a deflection bounded by $\pm\delta_{\max}(\delta_{\max} = 12^\circ)$, one tilting the thrust about the body-y axis (δ_{pitch}) and the other about the body-x axis (δ_{lat}), giving a body-frame thrust vector

$$\mathbf{F}_B = T \begin{bmatrix} \sin\delta_{\text{pitch}} \cos\delta_{\text{lat}} \\ -\sin\delta_{\text{lat}} \\ \cos\delta_{\text{pitch}} \cos\delta_{\text{lat}} \end{bmatrix}, \tag{1}$$

which is rotated into the inertial frame and applied at the engine attachment point. Roll is left uncontrolled, since *a centrally mounted single-gimbal engine*

Table 1: Physical and actuation parameters of the simulated booster.

Parameter	Symbol	Value
Mass	m	20 kg
Radius / Length	R, L	0.5 m, 3.0 m
Trans. / axial inertia	$I_{xx}{=}I_{yy}$, I_{zz}	16.25, 2.5 kg m^2
COM-to-engine offset	ℓ	1.5 m
Max. thrust	$T_{\max}$	300 N
Max. gimbal deflection	$\delta_{\max}$	12°
Max. control torque	$\tau_{\max}$	93.6 N m
Gravity	g	9.8 m s^{-2}
Hover throttle	$mg/T_{\max}$	0.65
Control rate	f	20 Hz

Table 2: Landing gate and episode limits.

Quantity	Limit
$\lvert v_z\rvert$	$< 1.5\,\mathrm{m\,s^{-1}}$
$\lvert v_x\rvert, \lvert v_y\rvert$	$< 1.0\,\mathrm{m\,s^{-1}}$
Ψ_{tilt}	< 0.30 rad
Pad offset	< 1.0 m
Tip-over	$\Psi_{\text{tilt}} > 0.5$ rad
Truncation	400 steps

generates no moment about its own thrust line, and is instead proportionally damped at each sub-step, replicating prior work in 6-DOF [10].

Rigid-body motion follows the standard Newton-Euler equations [11]:

$$m\,\dot{\mathbf{v}} = R\,\mathbf{F}_B + m\,\mathbf{g}, \qquad J\,\dot{\boldsymbol{\omega}} + \boldsymbol{\omega} \times (J\,\boldsymbol{\omega}) = \mathbf{r}_e \times \mathbf{F}_B, \tag{2}$$

Here, $R \in SO(3)$, parameterised as a unit quaternion to avoid gimbal-lock singularities, rotates body-frame vectors into the inertial frame, and $J = \mathrm{diag}(I_{xx}, I_{yy}, I_{zz})$. Since thrust acts at the fixed offset $\mathbf{r}_e = [0, 0, -\ell]^\top$, the transverse control torques reduce to $\tau_x = -\ell T \sin\delta_{\text{lat}}$ and $\tau_y = -\ell T \sin\delta_{\text{pitch}} \cos\delta_{\text{lat}}$, giving a maximum attainable torque $\tau_{\max} = \ell T_{\max} \sin\delta_{\max} \approx 93.6\,\mathrm{N\,m}$.

A landing is judged successful when the touchdown gate of Table 2 is satisfied at ground contact. Episodes otherwise terminate on a crash landing, a tip-over, or an out-of-bounds violation, and are truncated at 400 steps (20 s at $f = 20$ Hz).

Table 3: Initial-state distribution by curriculum level.

Parameter	Level 0	Level 1	Level 2	Level 3
z_0 (m)	$[1.7, 2.5]$	$[2.5, 4.0]$	$[4.0, 6.0]$	$[8.0, 12.0]$
x_0, y_0 (m)	$[-1.0, 1.0]$	$[-1.2, 1.2]$	$[-2.0, 2.0]$	$[-3.0, 3.0]$
v_{z0} (m · s^{-1})	$[-1.0, -0.3]$	$[-1.5, -0.4]$	$[-2.5, -0.8]$	$[-7.0, -3.0]$
v_{x0}, v_{y0} (m · s^{-1})	$[-0.5, 0.5]$	$[-0.8, 0.8]$	$[-1.2, 1.2]$	$[-4.0, 4.0]$
θ_0, ψ_0 (rad)	$[-0.05, 0.05]$	$[-0.07, 0.07]$	$[-0.12, 0.12]$	$[-0.25, 0.25]$

Following evidence that curriculum learning aids convergence under disturbance in related work [22], initial conditions are drawn from a four-level curriculum that progressively widens starting altitude, lateral offset, velocity, and

attitude perturbation (Table 3), allowing the agent to consolidate near-terminal landing behaviour facing high-altitude, high-velocity descents.

4 Baseline: Cascaded PD Controller

As a classical, non-learned baseline, we implement a cascaded proportional-derivative (PD) controller commanding five of the six rigid-body degrees of freedom – three translational and two lateral attitude channels, via the same gimballed engine described in Section 3. Three nested loops each generate the reference for the next: a *lateral position loop* converts horizontal position/velocity error into desired tilt angles $(\theta_{\text{des}}, \psi_{\text{des}})$; an *altitude loop* converts vertical position/ velocity error, compensated for tilt, into a collective thrust command T; and an *attitude loop* converts tilt and body-rate error into commanded moments (τ_x, τ_y), allocated to thrust and gimbal deflections as in Eq. (2). No integral term is used, as the gravity feed-forward mg analytically cancels the dominant steady-state disturbance, avoiding the windup and phase lag an integrator would introduce as the gimbal saturates. The commanded thrust T is divided by the tilt projection $R_{33}\cos\chi$ to compensate for thrust acting along the body rather than the inertial axis.

Gains elsewhere follow the *second-order law*, $K_p = m\omega_n^2,\ K_d = 2m\zeta\omega_n$ with I_{xx}/I_{yy} substituted for m on the attitude axes. All axes are overdamped ($\zeta = 1.2$) and the attitude loop is tuned four times faster than the outer lateral loop ($\omega_{\text{att}} = 4.0,\ \omega_{\text{alt}} = 1.5,\ \omega_{\text{lat}} = 1.0$ rad s^{-1}) so commanded tilt references settle on the outer loop's own timescale. Roll is left uncontrolled, with evaluation under identical termination and success criteria to the learned policies.

5 Markov Decision Process Problem Formulation

We formalise rocket landing as a Markov Decision Process (MDP) for the single-objective (SORL) baselines and as its multi-objective extension (MOMDP) for CAPQL, sharing an identical state and action space so that the two settings differ only in reward structure and learning algorithm.

State and action spaces. The 14-dimensional state

$$s = [x, y, z, \Delta v_x, \Delta v_y, \Delta v_z, q_x, q_y, q_z, q_w, \omega_x, \omega_y, \omega_z, \hat{t}_{\text{go}}]^\top$$

comprises horizontal and vertical positions, a velocity-tracking error $\Delta\mathbf{v} = \mathbf{v} - \mathbf{v}^*$ against a guidance-field target, a hemisphere-resolved attitude quaternion, world-frame angular velocity, and a clipped time-to-go estimate, respectively. Utilising the velocity *error* (Δv) against the analytical guidance field, rather than raw inertial velocity, reduces the exploration burden by directing the agent to correct deviations rather than track absolute velocity. The guidance target switches at $z_{\text{switch}} = 3.0$ m: above it, $\mathbf{v}^*$ decays exponentially toward the switching altitude; below it, it reduces to a proportional law centring the booster

horizontally while scaling vertical speed to zero at touchdown. The action space is $a = [a_0, a_1, a_2]^\top \in [-1, 1]^3$: collective throttle and two gimbal deflections, mapped after normalisation as in Section 3.

Single-objective reward. For the SORL baseline, the MDP is $(\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma)$ with transition kernel $\mathcal{P}$ realised by the PyBullet solver at 20 Hz, discount $\gamma = 0.99$, and a scalar reward $r = r_{\text{shaping}} + r_{\text{terminal}}$, where

$$\begin{aligned} r_{\text{shaping}} = & -0.1\sqrt{x^2 + y^2} - 0.1\,|z - 0.2| - 0.2\,\|\mathbf{v}\| \\ & - 5.0\,\Psi_{\text{tilt}} - 5.0\,\|\boldsymbol{\omega}\| + r_{\text{progress}} + r_{\text{attitude}} + r_{\text{approach}}, \end{aligned} \tag{3}$$

penalises lateral, altitude, speed, tilt, and angular-rate deviation using empirically tuned weights; r_{progress} rewards controlled descent over hovering; r_{attitude} escalates once tilt exceeds a warning threshold ahead of the hard tip-over limit; and r_{approach} tightens velocity-gated penalties once the vehicle enters the terminal region below z_{switch}. The sparse terminal term is $r_{\text{terminal}} = +100$ for a soft landing satisfying the touchdown gate of Section 3, and -100 for a failed termination. This structure follows standard practise in prior SORL rocket-landing work [4, 23, 10, 3].

Multi-objective reward. For CAPQL, the state and action spaces are unchanged, but the scalar reward is replaced by a vector $\mathbf{r} = [r_{\text{prec}}, r_{\text{fuel}}]^\top \in \mathbb{R}^2$. The precision channel mirrors the SORL shaping terms, with the tolerated descent speed shrinking as the booster nears the pad,

$$v_z^{\text{budget}} = v_z^{\max}\,(1 - \beta\, n)\,, \qquad n = \text{clip}\left(\frac{z_{\text{switch}} - z}{z_{\text{switch}} - z_{\text{td}}}, 0, 1\right), \tag{4}$$

to discourage reward hacking by hovering just above touchdown. The fuel channel penalises commanded throttle fraction directly, $r_{\text{fuel}}^{\text{shape}} = -\kappa_f\,\delta_T - c_f$ ($\kappa_f = 3.0$, $c_f = 0.05$). At termination, a shared penalty $r_{\text{fail}} \in \{0, -100\}$ is applied to *both* channels, so no preference weighting can construe crashing as fuel-saving. A quality-weighted success bonus $r_{\text{success}} = B_{\text{base}} + B_{\text{qual}} \cdot q$ where, $q \in [0, 1]$ scores how comfortably the vertical and horizontal speed, tilt, and position sit inside the landing gate, and is credited to the precision channel *only*:

$$\mathbf{r} = \left[\, r_{\text{prec}}^{\text{shape}} + r_{\text{success}} + r_{\text{fail}},\;\; r_{\text{fuel}}^{\text{shape}} + r_{\text{fail}} \,\right]^\top. \tag{5}$$

6 Training Configuration

All three RL algorithms are trained on the environment and reward formulations of Section 5; their specific network architectures, hyperparameters, and training setups are summarized in Table 4.

Table 4: Hyperparameter and architectural configurations across algorithms.

Parameter / Setting	**PPO**	**SAC**	**CAPQL**
Actor Architecture	[256, 256], Tanh	[256, 256], ReLU	[256, 256, 128], ReLU
Critic Architecture	[256, 256], Tanh	Twin [256, 256], ReLU	Ensemble $N_Q = 3$ [256, 256, 128]
Initialization	Orthogonal	Standard	Standard
Obs. Normalization	—	Online (`VecNormalize`)	Online (Shared Train/Eval)
Total Training Steps	3×10^6[a]	6×10^5	7.5×10^5
Buffer / Warmup	— (On-policy)	10^6 / 25,000	10^6 / 50,000
Batch Size	256 ($K = 6$ epochs)	256	256
Rollout Steps	4,096 ($N = 8$ envs)	1 per env step	1 per env step
Target Update	—	Polyak ($\tau = 0.005$)	Vector-bootstrapped ($\mathbf{w}^\top \mathbf{Q}$)
Entropy Tuning	—	Auto (Target -3)	Fixed ($\alpha = 0.1$)

[a] Collected across $N = 8$ parallel environments, equivalent to $\approx 4 \times 10^5$ timesteps per environment.

7 Results

7.1 Classical Baseline and Preliminary RL Results

Across 25,000 randomised trials per curriculum level (CL), the PD controller registers a reliable 78.5% landing rate at CL0. A mediocre 54.7% is recorded at CL1 before collapsing to 5.5% at CL2 and 0.0% at CL3, where every failures ends in a crash.

The single-objective baselines substantially outperform the PD controller, with PPO and SAC reaching landing rates of 91.0 ± 1.8% and 93.0 ± 1.6% and comparable mean returns (75.3 and 67.82), though PPO shows a wider spread in radial offset and angular displacement. Landing rates are given with 95% binomial confidence intervals over $n = 1{,}000$ evaluation episodes; all other metrics are mean ± SD. The multi-objective CAPQL agent exhibits a clear precision-fuel trade-off. CAPQL-P (precision favoured) achieves a landing rate of 92.9 ± 1.6%, alongside the *tightest radial offset* (0.11 ± 0.008 m), but at the cost of a steeper fuel penalty, while CAPQL-F (fuel favoured) improves fuel return (-31.08 ± 1.081) at the expense of precise landings (83.4 ± 2.3%) and touchdown dispersion (0.8 ± 0.420 m radial offset, 1.98 ± 1.489° angular displacement) (Table 5).

7.2 Multi-Objective Reinforcement Learning Results

After a 50,000 step random-action warmup and a brief destabilisation as the actor moves off the behaviour policy (at ≈ 91,000 steps), the episodic return

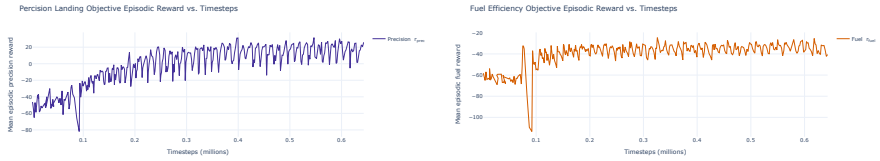
Table 5: Comparative performance of the PD baseline and learned policies recorded at curriculum level 3 (single seed, evaluated at 1,000 episodes per policy for RL).

Agent	Objective	Mean Return	Landing Rate (95% CI)(%)	Radial Offset (m)	Angular Disp. (Landings)(deg)
PD Baseline	—	—	0.0	—	—
PPO	—	75.3 ± 8.136	91.0 ± 1.8	0.32 ± 1.064	1.24 ± 1.03
SAC	—	67.82 ± 0.491	93.0 ± 1.6	0.72 ± 0.203	0.78 ± 0.282
CAPQL-P[a]	Precision	27.96 ± 1.888	92.9 ± 1.6	0.11 ± 0.008	0.9 ± 0.265
	Fuel	-40.28 ± 2.026			
CAPQL-F[b]	Precision	1.02 ± 4.875	83.4 ± 2.3	0.8 ± 0.420	1.98 ± 1.489
	Fuel	-31.08 ± 1.081			

Legend: CI = Confidence Interval.
Note: **Initial state domain randomisation** was used for all RL agents.

[a] Precision-favoured preference weighting.
[b] Fuel-favoured preference weighting.

rises near-monotonically for both the precision (Figure 1a) and the fuel objectives (Figure 1b). The violent oscillations observed connote varying per-objective returns as a consequence of preference sampling at training time.

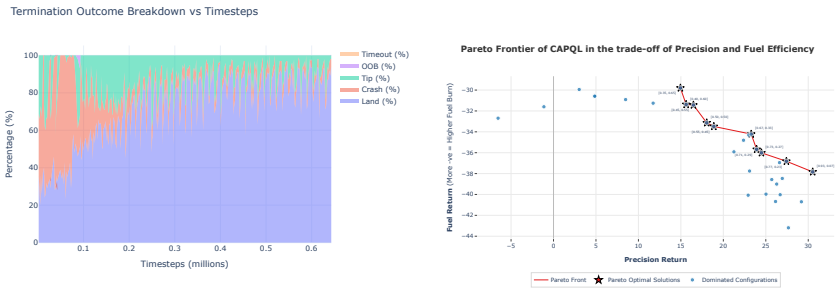


(a) Precision objective r_{prec} (b) Fuel objective r_{fuel}

Fig. 1: Per-objective mean episodic reward vs. timesteps for CAPQL

Crashes and tip-overs dominate the initial termination (Figure 2a); successful landings supersede failed terminations as training continues to convergence. Across thirty preference weights, the converged policy reveals a non-uniform trade-off: precision return rises from ≈-4 to ≈15 as the precision weight increases from 0 to 0.4 while the fuel return stays within 1.5 units of its maximum, but beyond $w_{\text{prec}} \approx 0.6$ fuel degrades steeply to ≈ -43.5 at the precision extreme. The operational trade-off point sits near $w_{\text{prec}} \approx 0.5$ - 0.6, not the midpoint, since the shared failure penalty bounds how far the policy can profitably chase fuel savings. Ten of the thirty evaluated weights are non-dominated, tracing a near

smooth, concave Pareto front from (15.0, -29.8) to (30.5, -37.8) (Figure 2b). Recovering the final ≈ 15 units of precision costs ≈ 8 units of fuel, with the exchange rate steepening toward the precision extreme.



(a) Termination-outcome breakdown (b) Approximate Pareto front

Fig. 2: CAPQL performance metrics: (a) Outcome breakdown vs. timesteps, and (b) precision-fuel trade-off showing non-dominated solutions (stars).

7.3 The MORL Advantage

The advantage MORL demonstrates is structural rather than a raw performance edge. On landing rate alone, all three methods are broadly comparable, with PPO and SAC converging to 87.12 - 94.85% and CAPQL's evaluation sweep spanning 81.5 - 95.1% depending on the assessed preference weight. The distinction is that PPO and SAC each commit, at training time, to one fixed, irreversible policy, whereas CAPQL yields a single set of weights that, conditioned on a preference vector at inference time, reproduces the entire spectrum of trade-offs between landing precision and propellant use – a property a scalarised baseline cannot even be tested for, since it was never trained to represent more than one point on that spectrum. For a task whose objectives are genuinely in tension, MORL therefore recovers more decision-relevant information than its single-objective counterpart, at parity of landing competence and without the engineering burden of retraining a new policy each time mission priorities shift.

8 Conclusion

This paper investigated whether 6-DOF rocket booster landing, formalised as a multi-objective problem, can be solved by Multi-Objective Reinforcement Learning (MORL) to yield a useful family of trade-off policies rather than a single arbitrary compromise. A custom 6-DOF PyBullet environment modelling full rigid-body dynamics, gimballed thrust vectoring, and physical landing constraints

was developed for this study. A fixed-gain PD controller proved insufficient for the coupled attitude-position regulation across the curriculum levels, motivating a learned approach. Two single-objective baselines, PPO and SAC, reached landing rates of 91.0% and 93.0%, SAC doing so an order of magnitude more sample-efficiently via off-policy replay, though each committed at training time to a single fixed reward weighting.

Preference-conditioned CAPQL removed this inflexibility, recovering a Pareto front of ten non-dominated solutions; across the preference sweep, landing rates ranged from 83.4% (fuel-favoured) to 92.9% (precision-favoured), precision and fuel returns traded off smoothly, and a shared failure penalty precluded crashing to conserve fuel. These results confirm MORL's applicability to 6-DOF rocket landing and its advantage over the single-objective paradigm that dominates prior work: one arbitrary policy is replaced by an explicit, selectable menu of trade-offs, without sacrificing landing competence.

This work is confined to simulated powered descent, omitting ascent, staging, re-entry, atmospheric disturbances, and depleting propellant mass. Hypervolume and expected utility metric were computed but are deferred to an extended version of this work. Future work will target sim-to-real transfer, higher-fidelity dynamics, more systematic reward and curriculum design, and hybrid architectures pairing classical control for certifiable low-level stability with RL for high-level trajectory and preference optimisation.

Disclosure of Interests. There exists no competing interests to declare that are relevant to the content of this article.

Acknowledgments. This publication has been supported in part by the Science Foundation Ireland under Frontiers for the Future Grant No. 21/FFP-A/8957. For the purpose of Open Access, the authors have applied a CC BY public copyright license to any Author Accepted Manuscript version arising from this submission.

References

1. Açıkmeşe, B., Carson, J.M., Blackmore, L.: Lossless convexification of nonconvex control bound and pointing constraints of the soft landing optimal control problem. IEEE Transactions on Control Systems Technology **21**(6), 2104–2113 (2013)
2. Alkodur, M.A., Almusharraf, A.H., Abro, G.E.M., Mahmoud, E., Abdallah, A.M.: Exploring pid and fopid control methods for thrust vectoring in aerospace systems-a beginner's guide. IEEE Access **13**, 41270–41281 (2025)
3. Chen, D., Wu, D., Cheng, L., Gong, S.: Robust model-based reinforcement learning for rocket landing. IEEE Trans. on Aerospace and Electronic Systems **62** (2026)
4. Chowdhury, A., Joshi, P., Awasthi, V., Tripathi, N.: Autonomous control for reusable rocket landing: Deriving a robust guidance through deep reinforcement learning in a custom 3d simulation environment. In: Proc. (ICAART). vol. 3, pp. 2785–2792. SciTePress (2026)
5. Çıldır, M.E., Doğru, M.H., Öztürk, E.: Comparative study of PID and MPC controllers for vertical take-off and vertical landing (VTVL) rocket control. The International Journal of Energy & Engineering Sciences **8**(2), 28–39 (2023)

6. Coumans, E., Bai, Y.: Pybullet, a python module for physics simulation for games, robotics and machine learning. `http://pybullet.org` (2016–2021)
7. De Almeida Souza, G., Silva, O.M., Maximo, M.R.O.A.: Thrust vectored rocket landing integrated guidance and control with proximal policy optimization (2022)
8. Felten, F., et al.: A toolkit for reliable benchmarking and research in multi-objective reinforcement learning. In: Proc. (NeurIPS 2023) (2023)
9. Gadgil, S., Xin, Y., Xu, C.: Solving the lunar lander problem under uncertainty using reinforcement learning (2020). https://doi.org/10.48550/arXiv.2011.11850
10. Gaudet, B., Linares, R., Furfaro, R.: Deep reinforcement learning for six degree-of-freedom planetary landing. Advances in Space Research **65**(7), 1723–1741 (2020)
11. Goldstein, H., Poole, C.P., Safko, J.L.: Classical Mechanics. Addison-Wesley, San Francisco, CA, 3 edn. (2002)
12. Hayes, C.F., et al.: A practical guide to multi-objective reinforcement learning and planning. Autonomous Agents and Multi-Agent Systems **36**(1), 26 (2022)
13. He, X., Lv, C.: Toward personalized decision making for autonomous vehicles: A constrained multi-objective reinforcement learning technique. Transportation Research Part C: Emerging Technologies **156**, 104352 (2023)
14. Hribar, J., Hackett, L., Dusparic, I.: Deep w-networks: Solving multi-objective optimisation problems with deep reinforcement learning. In: Proc. of the 15th International Conference on Agents and Artificial Intelligence. p. 17–26. SCITEPRESS - Science and Technology Publications (2023)
15. Lu, H., Herman, D., Yu, Y.: Multi-objective reinforcement learning: Convexity, stationarity and pareto optimality. In: The Eleventh International Conference on Learning Representations (2023)
16. Nugroho, L., Zani, N.R., Qomariyah, N., Akmeliawati, R., Andiarti, R., Wijaya, S.K.: Powered landing guidance algorithms using reinforcement learning methods for lunar lander case. Indonesian Journal of Aerospace **19**(1), 43–56 (2021)
17. Rosero, J.C., Dusparic, I., Cardozo, N.: Multi-objective deep reinforcement learning for optimisation in autonomous systems (2024). https://doi.org/10.1109/ACSOS-C63493.2024.00038
18. Space Exploration Technologies Corp.: Falcon User's Guide. Space Exploration Technologies Corp. (March 2025), `https://www.spacex.com/assets/media/falcon-users-guide-2025-05-09.pdf`, version 8
19. Sutton, R.S., Barto, A.G.: Reinforcement learning: An introduction. The MIT Press, Cambridge, Massachusetts, 2nd edn. (2018)
20. Towers, M., et al.: Gymnasium: A standard interface for reinforcement learning environments (2024)
21. Wang, J., Cui, N., Wei, C.: Optimal rocket landing guidance using convex optimization and model predictive control. JGCD **42**(5), 1078–1092 (2019)
22. Yuan, H., Zhao, Y., Mou, Y., Wang, X.: Leveraging curriculum reinforcement learning for rocket powered landing guidance and control. In: 2021 China Automation Congress (CAC). pp. 5765–5770 (2021)
23. Zhu, J.: AdaptPPO: An adaptive proximal policy optimization algorithm for precision rocket landing simulation. In: Proceedings of the 2025 International Conference on Computers, Information Processing and Advanced Education (CIPAE). pp. 307–313. IEEE (2025)